

BloodHound: el grafo de permisos

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

BloodHound recolecta usuarios, grupos, equipos, sesiones y ACLs del dominio y los representa como un grafo. En lugar de adivinar que permiso abusar, ves la ruta concreta: quien puede resetear la password de quien, que grupo tiene GenericAll sobre otro, donde hay sesiones de admin que robar.

En un dominio real las relaciones de permisos son miles y a mano es imposible. El grafo revela caminos no obvios (encadenar tres permisos mediocres para llegar a DA) que definen el ataque en AD (ad-modelo).

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

En cuanto tengas UNA credencial valida. Es lo que haces cuando el roasting (asrep-kerberoast) no basto o para planificar el salto final a Domain Admin.

Cómo identificarla

- Aristas 'GenericAll', 'WriteDACL', 'ForceChangePassword' hacia grupos privilegiados.
- Sesiones de un admin en una maquina donde tu tienes acceso.
- Un camino corto marcado hacia el grupo Domain Admins.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Corre bloodhound-python con la credencial y confirma que el .zip se genera sin errores de autenticacion.

Evidencia esperada: El .zip se importa en la GUI y aparece tu usuario en el grafo; 'Shortest paths to Domain Admins' muestra al menos una ruta.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.

4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
bloodhound-python -d $DOMAIN -u $USER -p $PASS -c all -ns $IP --zip
```

Objetivo y contexto: Recolecta todo el grafo del dominio con una sola credencial, sin necesidad de pisar una maquina Windows. -c all junta toda la informacion; --zip lo deja listo para importar.

Resultado esperado: Un .zip para arrastrar a la GUI de BloodHound. Allí marca tu usuario como owned y lanza la consulta de rutas a Domain Admins.

Anatomía de los payloads

La primera prueba es `bloodhound-python -d $DOMAIN -u $USER -p $PASS -c all -ns $IP --zip`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

El .zip se importa en la GUI y aparece tu usuario en el grafo; 'Shortest paths to Domain Admins' muestra al menos una ruta.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Recolecta los datos del dominio con un colector (bloodhound-python o SharpHound).
- Importa el zip en la GUI y marca tu usuario como 'owned'.
- Usa 'Shortest paths to Domain Admins' y estudia cada arista del camino.
- Abusa del permiso concreto (cada tipo de arista tiene su tecnica) y avanza un paso.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- No tienes ninguna credencial de dominio todavía: BloodHound necesita autenticarse para recolectar.
- El dominio es trivialmente pequeño (pocos usuarios/equipos) y ya ves el camino a mano.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Mapea el dominio como un grafo y te muestra la ruta mas corta de tu usuario a Domain Admin. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (bloodhound)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.